



Tutorial: Geospatial Usage

Geospatial libraries such as `geopandas`, `shapely` and `fiona` are available for working with spatial data and operations in Python. One way to work with spatial data in Databricks is to wrap functions from those libraries in an user-defined function in Spark.

However, to make full use of Spark's distributed computing capabilities, we can also use [Apache Sedona](#), which is an extension providing PostGIS functionality to Apache Spark.

Tutorial notebook

This [notebook](#) demonstrates the use of `OneMap`, `Apache Sedona`, `geopandas` and `folium` on DataHive Databricks with a sample geospatial problem statement on `lta_taxi` data.

The tutorial goes over the following:

1. Querying OneMap for location data
2. Creating a 200m buffer with coordinate transformations and dissolving the points
3. Load dataset, filter spatially and temporally
4. Visualise with heatmap in `folium`
5. Plot pickup and drop-off counts within buffered region over time

Registering Apache Sedona

To register Apache Sedona functions in Databricks, use the following 2 lines:

```
from datahive.geospatial import register_sedona
register_sedona()
```

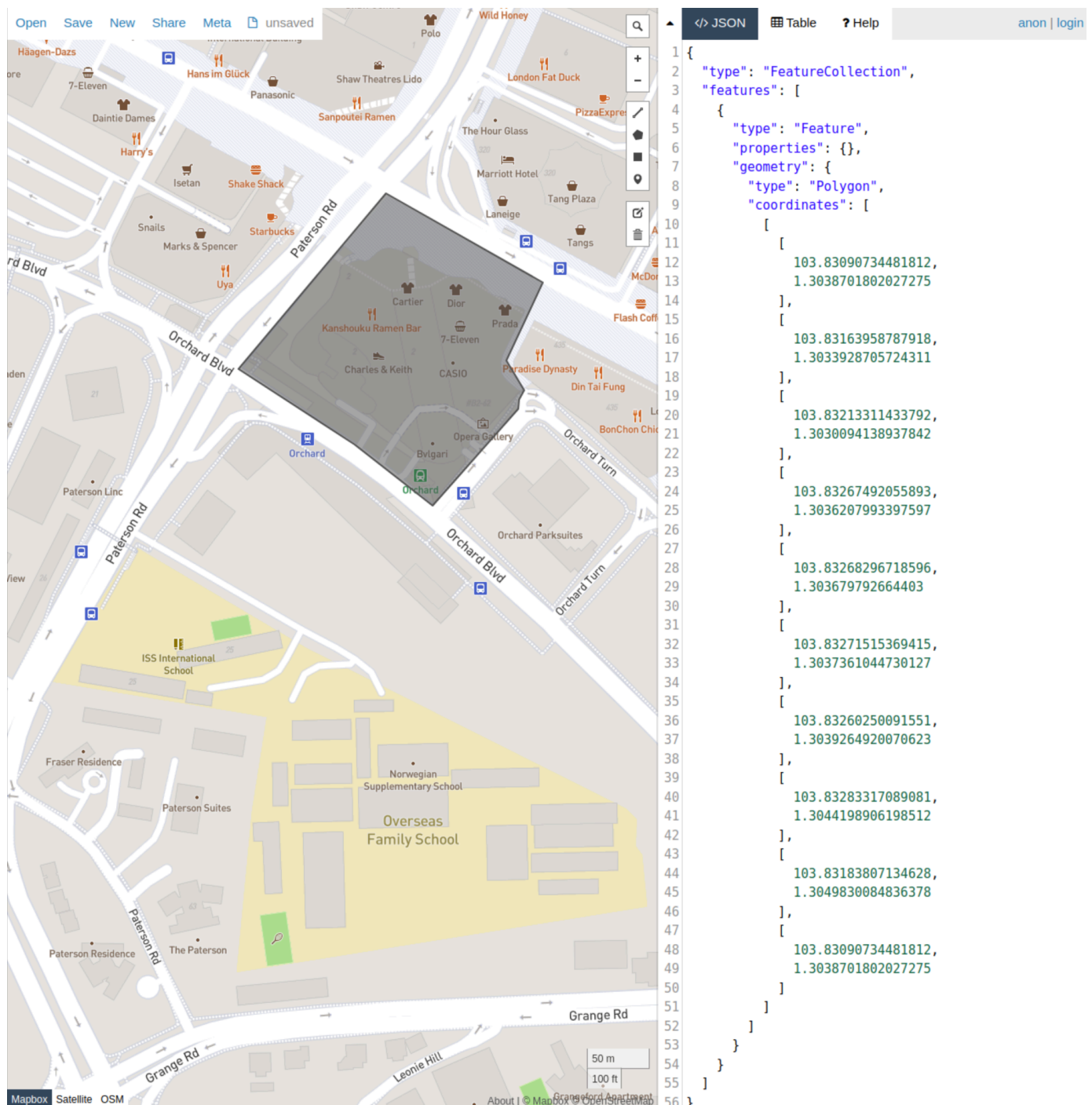
NOTE

Registration is only required when using the Apache Sedona Python API (e.g. `GeoDataFrame.pipe(spark.createDataFrame())`).

All the Apache Sedona `ST_*` SQL functions have already been pre-registered via cluster init-scripts.

Create custom GeoJSON and import into Databricks

In addition to querying points and creating buffers as shown in the tutorial, you can use geojson.io to manually define an area of interest on the map. The JSON in the text box can be copied into the Databricks UI for import as a Spark DataFrame using the DataHive SDK.



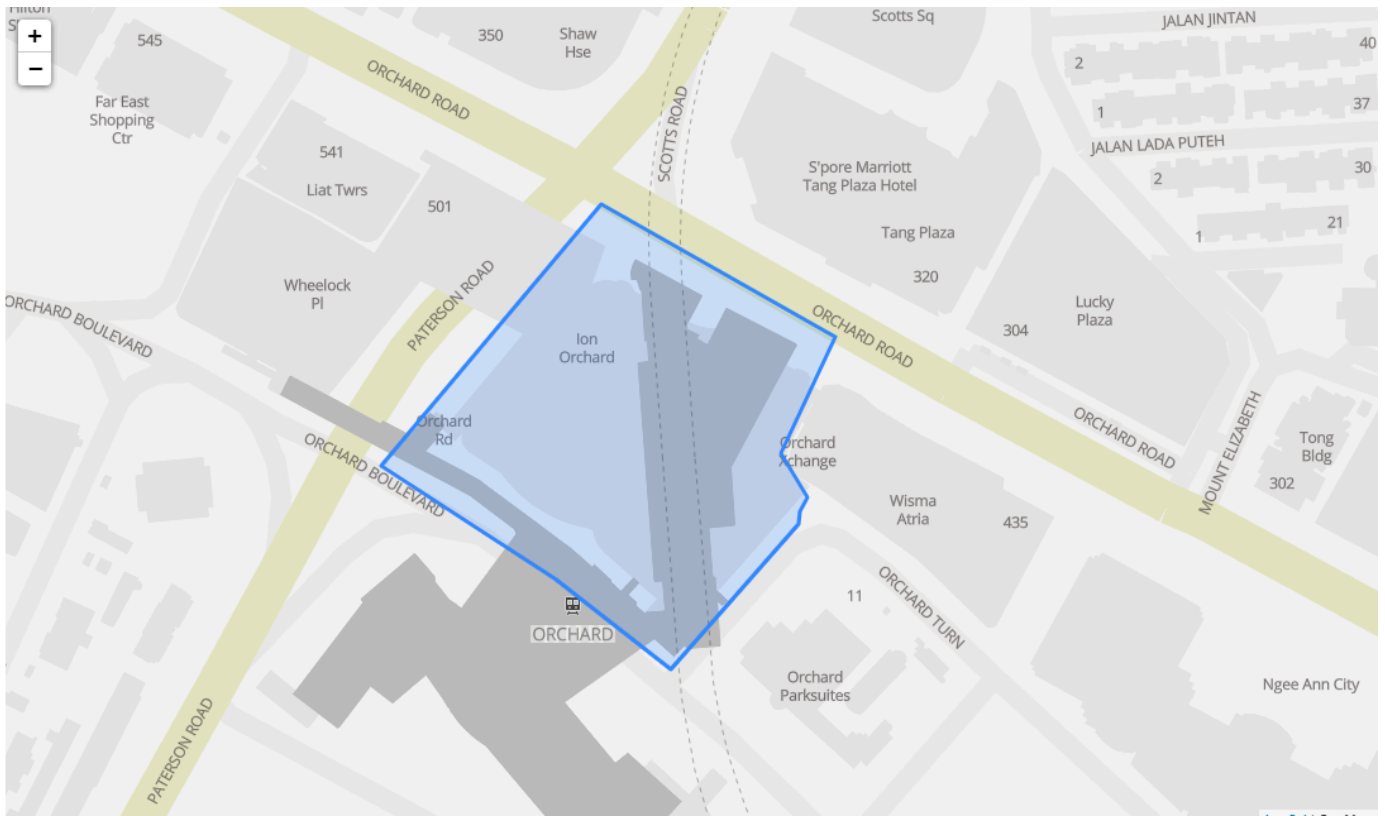
The screenshot displays a map interface with a polygon drawn on a street map of Orchard Road, Singapore. The polygon is dark grey and covers several buildings and streets. The map includes labels for various locations like Cartier, Dior, Prada, and 7-Eleven. The interface also shows a JSON editor on the right side with a pre-defined GeoJSON structure.

```
1 {
2   "type": "FeatureCollection",
3   "features": [
4     {
5       "type": "Feature",
6       "properties": {},
7       "geometry": {
8         "type": "Polygon",
9         "coordinates": [
10          [
11            [
12              103.83090734481812,
13              1.3038701802027275
14            ],
15            [
16              103.83163958787918,
17              1.3033928705724311
18            ],
19            [
20              103.83213311433792,
21              1.3030094138937842
22            ],
23            [
24              103.83267492055893,
25              1.3036207993397597
26            ],
27            [
28              103.83268296718596,
29              1.303679792664403
30            ],
31            [
32              103.83271515369415,
33              1.3037361044730127
34            ],
35            [
36              103.83260250091551,
37              1.3039264920070623
38            ],
39            [
40              103.83283317089081,
41              1.3044198906198512
42            ],
43            [
44              103.83183807134628,
45              1.3049830084836378
46            ],
47            [
48              103.83090734481812,
49              1.3038701802027275
50            ]
51          ]
52        ]
53      }
54    ]
55  }
```

To import the GeoJSON:

```
from datahive.geospatial import read_geojson_raw

ion_geojson = """
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "properties": {},
      "geometry": {
        "type": "Polygon",
        "coordinates": [
          [
            [
              103.83090734481812,
              1.3038701802027275
            ],
            [
              103.83163958787918,
              1.3033928705724311
            ],
            [
              103.83213311433792,
              1.3030094138937842
            ],
            [
              103.83267492055893,
              1.3036207993397597
            ],
            [
              103.83268296718596,
              1.303679792664403
            ],
            [
              103.83271515369415,
              1.3037361044730127
            ],
            [
              103.83260250091551,
              1.3039264920070623
            ],
            [
              103.83283317089081,
              1.3044198906198512
            ]
          ]
        ]
      }
    ]
  }
}
```

⚠ CAUTION

If the geojson file you are importing is too large, the Databricks UI might crash. We plan to add functionality for uploading GeoJSON files soon.

Resources

- [Apache Sedona SQL](#): spatial queries in Spark
- [folium](#): visualizing spatial data on an interactive map